JIT (Just-in-Time) Tool Spawning Protocol: A Server-Side Routing Architecture for Infinite-Tool AI Agents

Akshat Dwivedi
Founder & CEO, Sashvat Bharat

March 2026

Abstract

For years, the AI community has relied on a brute-force approach to multi-agent systems: stuffing the definitions, schemas, and parameters of every available tool directly into an LLM's system prompt. This method causes massive API costs, severe latency spikes, and degraded reasoning (the "Lost in the Middle" phenomenon).

This paper introduces the JIT (Just-in-Time) Tool Spawning Protocol, a highly optimized backend architecture. Instead of front-loading context, this protocol stores tool schemas in a vector database, uses semantic similarity to fetch a Top 5 list, and employs a high-speed Small Language Model (SLM) to filter the exact tools needed based on user intent. By dynamically injecting only 1 to 3 requisite tools into the primary LLM, systems can scale infinitely while reducing compute overhead significantly

our experiments show up to ~88.8% token savings compared to the traditional approach.

Source Code & Implementation: The reference implementation of the JIT Tool Spawning Protocol is open-sourced and available at: https://github.com/sashvat-bharat/jit-tool-protocol

1. The Core Problem: Context Bloat

When building an enterprise AI agent with access to APIs, databases, and internal functions, the main Large Language Model (LLM) needs to know how to use them. The standard industry practice looks like this:

```JSON
// The old, broken method: injecting everything upfront.
SYSTEM PROMPT:
"You are a helpful assistant. Here are your 150 tools:
1. { full_json_schema_for_maps... }
2. { full_json_schema_for_weather... }
...
3. { full_json_schema_for_database_writes... }"
```

The Fallout:

1. **Financial Burning:** You pay for thousands of tokens on every single query, even if the user just said "Hello."
2. **Latency:** Processing massive system prompts severely slows down the Time-to-First-Token (TTFT).
3. **Hallucinations:** Overloaded LLMs frequently confuse parameters between similar tools because the attention mechanism is stretched too thin.

---

2. The JIT Protocol Architecture:

To achieve high-accuracy routing with minimal token usage, the JIT Protocol implements a strict three-tier server-side pipeline. It decouples the *discovery* of a tool from the *execution* of a tool.

Step 2.1: Vector Indexing (The Knowledge Base)

Before the user ever interacts with the system, we separate the "Searchable Description" from the "Execution Schema." We store the tools in a Vector Database (e.g., pgvector, Pinecone) by embedding a highly detailed, non-technical description of what the tool does.

```JSON
// Stored in Vector Database
"fetch_transactions": {
    "short_name": "fetch_transactions",
    "detailed_desc": "Used to retrieve historical financial transactions, ledger entries, and payment history for a specific company or user account."
}
```

Step 2.2: User Query & Coarse Retrieval (The Top K)

1. **Input:** The user states, *"Analyze the last 5 transactions of Sashvat Bharat."*
2. **Embedding:** The backend converts this query into a vector embedding using a fast-embedding model (e.g., gemini-embedding-001).
3. **Similarity Search:** The system runs a Cosine Similarity search against the Vector DB, returning the **Top K** closest matches (e.g., fetch_transactions, delete_transaction, get_user_profile, etc.).

> **Note on Top K:** In our reference implementation, we use a default of K=5, which provides a strong balance between recall and precision. However, this value is fully configurable — developers should adjust K based on their total tool count, tool similarity density, and performance requirements. A system with 500+ highly similar tools may benefit from K=7 or K=10, while a smaller registry with distinct tools could work well with K=3.

Step 2.3: SLM Semantic Routing (The Fine Filter)

Vector DBs rely on *semantic similarity*, not *logical intent*. A search for "transactions" will blindly pull up both fetch_transactions and delete_transaction. Giving the main LLM the wrong menu is dangerous.

To bridge this gap, we pass the Top K results through a lightning-fast Small Language Model (SLM) like **gemini-3-flash-preview** or **GPT-5-nano** etc.

The SLM Prompt:

> **SYSTEM:** You are a routing agent. Look at the user query and the 5 available tools. Return ONLY the exact name of the required tool in a JSON array. Do not answer the user's question.
>
> **AVAILABLE TOOLS:**
> - fetch_transactions
> - delete_transaction
> - get_user_profile
>
> **USER:** Analyze the last 5 transactions of Sashvat Bharat.

SLM Output (Latency ~200ms):
["fetch_transactions"]

> **On SLM Accuracy:** The accuracy of this routing step depends on the SLM chosen. Off-the-shelf models like gemini-3-flash-preview already perform extremely well on this narrow classification task. For teams requiring even higher precision, fine-tuning a small open-source model (3–4B parameters) specifically on tool-routing data can further reduce the misclassification rate to near zero, giving you a dedicated routing engine tailored to your exact tool set.

Step 2.4: Main LLM Injection & Execution

Now that the SLM has isolated the necessary tool with clear intent, the backend fetches the **massive, heavy JSON parameter schema** for fetch_transactions from a static registry and injects it into the Heavy LLM (e.g., Gemini 3 Pro).

The Final Payload sent to the Main AI:

```markdown
SYSTEM:
You are an expert financial analyst. You have access to the following tool to
fulfill the user's request.
Execute it strictly according to the schema:

{
  "name": "fetch_transactions",
  "description": "Fetch transaction history.",
  "parameters": {
    "type": "object",
    "properties": {
       "company_name": {"type": "string"},
       "limit": {"type": "integer"}
    },
    "required": ["company_name", "limit"]
  }
}

USER:
Analyze the last 5 transactions of Sashvat Bharat.
```

Result: The Heavy LLM executes the tool flawlessly with near-zero context bloat.

---

3. The Mathematical Advantage

By routing through an SLM before hitting the expensive Heavy LLM, the token cost flattens entirely, regardless of how many tools are in your database.

$$Cost_{JIT} = Cost_{Fast_Router} + [Rate_{Heavy_LLM} \times (Tokens_{Prompt} + Tokens_{1_Needed_Tool})]$$

Whether your agent has 10 tools or 10,000 tools, the operational cost per query remains mathematically bounded and predictable.

Experimental Results:

Metric	Traditional Approach (50 Tools)	JIT Tool Spawning Protocol
Tool definitions sent to Heavy LLM	All 50 schemas	Only 1–3 selected schemas
Tokens consumed for tools	**~5,000 tokens**	**~0 tokens** (handled by SLM)
SLM routing cost (Top 5 short descriptions + system prompt)	N/A	~500 tokens (tools) + ~60 tokens (system prompt) = **~560 tokens**
Total tool-related token overhead	**~5,000 tokens**	**~560 tokens**
Token savings	—	**~4,440 tokens (~88.8% reduction)**

This is with only 50 tools. As the tool registry scales to 200, 500, or 10,000+ tools, the traditional approach's cost grows linearly while the JIT Protocol's cost stays flat - the SLM always only sees the Top K short descriptions, no matter how large the registry becomes.

> **On Latency:** The JIT Protocol adds an extra routing step (~200ms for the SLM call) compared to the traditional single-call approach. This is a deliberate trade-off — you invest a small amount of latency upfront in the routing layer, but in return you save significant processing time on the Heavy LLM by feeding it a drastically smaller prompt. In systems with large tool registries, the net latency is often lower because the Heavy LLM responds faster with a leaner context.

Context Token Overhead

Traditional vs JIT Protocol (50 Registered Tools)

SASHVAT BHARAT RESEARCH • EXPERIMENTAL RESULTS FEB 2026
*THESE ARE THE ESTIMATES, REAL READINGS MAY VARY.

4. Handling Multi-Turn Conversations

A common question is: *how does this protocol handle follow-up queries?*

Consider this conversation:

1. **User:** *"What is the current trading price of Apple?"* → JIT routes to get_stock_price
2. **User:** *"Now email that to my boss."* → JIT routes to send_email

The second query on its own doesn't mention stocks at all. So how does the system know *what* to email?

The answer lies in the **memory layer**, which is a standard practice in modern agentic architectures. Most production AI systems already maintain a rolling conversation context — typically the last 2–3 exchanges — which gets loaded programmatically alongside the new user prompt. So, when the user says *"email that to my boss,"* the memory layer ensures the Heavy LLM already has the context of the previous stock price response.

The JIT Protocol works seamlessly with this pattern. The memory layer gives the system enough context to understand what *"that"* refers to, and the JIT router independently resolves that the user now needs the send_email tool. Each layer does its own job — memory handles *what*, and JIT handles *which tool*.

---

5. Conclusion

By separating tool discovery from tool execution, the JIT Tool Spawning Protocol allows AI systems to scale to thousands of capabilities without suffering from context collapse or spiraling API costs. Our experiments demonstrate an **88.8% reduction in tool-related token overhead** with just 50 tools — a saving that only grows larger as the tool registry scales.

It provides a mathematically sound, highly efficient foundation for the next generation of enterprise agentic architectures.